

Supplementary - Deep Embedded Non-Redundant Clustering

Lukas Miklautz,^{*,1} Dominik Mautz,^{*,2} Muzaffer Can Altinigneli,^{1,2} Christian Böhm,^{2,3} Claudia Plant^{1,4}

¹Faculty of Computer Science, University of Vienna, Vienna, Austria

²Ludwig-Maximilians-Universität München, Munich, Germany

³MCML, ⁴ds:UniVie

¹{lukas.miklautz,claudia.plant}@univie.ac.at

²{altinigneli,boehm,mautz}@dbs.ifi.lmu.de

Abstract

This document adds further information about the training setup and supplementary material like images for the data sets presented in the paper and a table with the used notation in Table 1. The accompanying Python code of ENRC and the used data sets can be found at <https://gitlab.cs.univie.ac.at/lukas/enrcpublic>.

1 Autoencoder Training

We utilize PyTorch (Paszke et al. 2017) machine learning framework and Fastai library (Howard and others 2018) for neural network training. We refactored PyTorch basic-blocks into more abstract ENRC sequential-building-blocks to improve code readability and to avoid coding mistakes. We illustrate the encoder’s building blocks in Figure 1¹, in top-down increasing abstraction levels. We depict PyTorch’s basic-blocks in white color and ENRC sequential-building-blocks in various other colors. The *Conv2d* applies two dimensional convolution over an image grid composed of several input channels and scales it down with a stride parameter that is greater than one. We employ the Rectified Linear Units *ReLU* introduced in (Nair and Hinton 2010) to mitigate the vanishing gradient problem and their leaky variants to prevent dying *ReLU*’s respectively. We use two dimensional batch norm blocks *BatchNorm2d* (Ioffe and Szegedy 2015) to accelerate training by using higher learning rates. The two dimensional dropout block *Dropout2d* in (Srivastava et al. 2014) prevents the layer’s neurons overfitting on certain inputs. The encoder’s convolutional layers contain *conv_and_res* blocks with different depths or number of filters.

We design the decoder similarly as illustrated in Figure 2. The *ConvTranspose2d* block in the decoder acts like the inverse operation of the *Conv2d* block in the encoder. The *ConvTranspose2d* scales up the image grid by padding zeros in-between existing pixels and learns the convolutional weights for the up-scaled image. The decoder’s inverse convolutional layers contain *conv_trans_and_res* blocks with

Table 1: Most important symbols

Symbol	Interpretation
$D \in \mathbb{N}$	Dimensionality of the embedded space
$S \in \mathbb{N}$	Number of clusterings
$K_s \in \mathbb{N}$	Number of Clusters in the s ’th clustering
$\mathcal{C}_{s,i}$	Objects of cluster i in clustering s
$\mathbf{x}$	A data point or object of the data set
z	A data point embedded by $\text{enc}(\mathbf{x})$
$\mu_{s,i} \in \mathbb{R}^d$	Cluster center of cluster i of clustering s
$V \in \mathbb{R}^{d \times d}$	Linear transformation initialized as an orthogonal matrix (rigid transformation)
$\beta_s \in \mathbb{R}^d$	Feature weights of clustering s

different depths or number of filters. In order not to clutter the figures, we omit a few PyTorch blocks, such as the *Upsample* block, which is the first block of the decoder applied after the last *AdaptiveAvgPool2d* block of the encoder.

The optimal maximum learning rate is determined with Fastai’s LR-Finder over a range of potential learning rates. Both the learning rate and the momentum of Adam optimization (Kingma and Ba 2014) are annealed based on (Smith 2015; Smith and Topin 2017; Smith 2018) during batch-wise training as illustrated in Figure 3. Our goal is to train our neural network faster than compared to the standard methods, so called super-convergence using cyclic learning in one-cycle. We present ENRC’s network layer and training settings with corresponding data sets in Table 2. The number of input pixels are given in PyTorch [width x height x channels] order. The number of decoder filters at each layer is organized in the reverse order of number of encoder filters, therefore is left out. For all data sets the same hidden layer (hl) dimensionality of 16 was used.

2 Datasets

2.1 NR-Objects

In this section we show a sample of images from the used data sets and their respective autoencoder reconstructions. In Figure 4 we show objects from the NR-Objects data set. To generate the images, we used the publicly available ren-

^{*}First authors with equal contribution.

Copyright © 2020, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

¹All figures are best viewed in color.

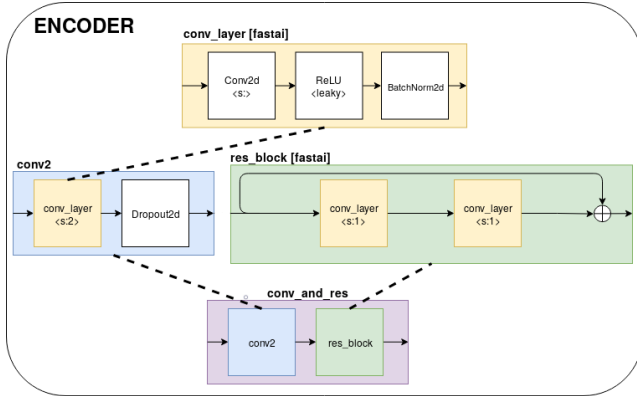


Figure 1: The architecture of Encoder. We depict PyTorch’s basic-blocks in white color and ENRC sequential-building-blocks in various other colors. $\langle s \rangle$ indicates stride and $\langle leaky \rangle$ indicates leakage-slope respectively.

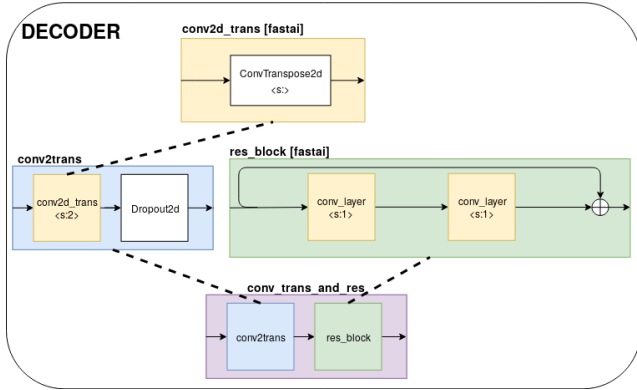


Figure 2: The architecture of Decoder is very similar to the architecture of Encoder.

Table 2: Summary of Network Layer and Training Settings. L2 Regularization with weight-decay= $1e-3$. Filters were chosen based on number of color channels. C-MNIST & Stickfigures are only greyscale, whereas NR-Objects & GTSRB are RGB images.

Name	# Input Pixels	# Encoder Filters	hl;dropout rate;leaky
C-MNIST	56x56x1	[8,16,32,64,128]	16;0.1;0.1
NR-Objects	128x128x3	[16,32,64,128,256]	16;0.3;0.1
GTSRB	32x32x3	[16,32,64,128,256]	16;0.1;0.1
Stickfigures	32x32x1	[8,16,32,64,128]	16;0.2;0.1

dering software², which was used in (Johnson et al. 2017). Each object can be clustered by three shapes, two materials and six colors.

2.2 C-MNIST

In Figure 5 the images and reconstructions of the independently concatenated MNIST digits (LeCun et al. 1998) is depicted. With this extension we simulate positional non-redundancy. This data set can be seen as containing two-digit numbers, from 00 to 99, where each digit (left and right) is independent from the other. We added black row wise pixels, so the new format is 56×56 instead of 28×56 , which would result from only concatenating the 28×28 MNIST images.

2.3 GTSRB

In Figure 6 a subset of the used images from the German Traffic Sign Benchmark (GTSRB) data set (Houben et al. 2013) are shown. The images have been preprocessed with histogram equalization to account for the low contrast of some of the images. We used a non-redundant subset of 4 different traffic signs: ‘Speed limit (70km/h)’, ‘No passing’, ‘Ahead only’, ‘Keep right’, which can be non-redundantly clustered w.r.t. to the four types of traffic sign and their two colors.

2.4 Stickfigures

The stickfigures data set contains nine basic objects of dancing figures. A subset of these object is depicted in Figure 7. It contains three clusterings for the upper and three for the lower body pose, a more detailed explanation can be found in (Ye et al. 2016).

²<https://github.com/facebookresearch/clevr-dataset-gen>

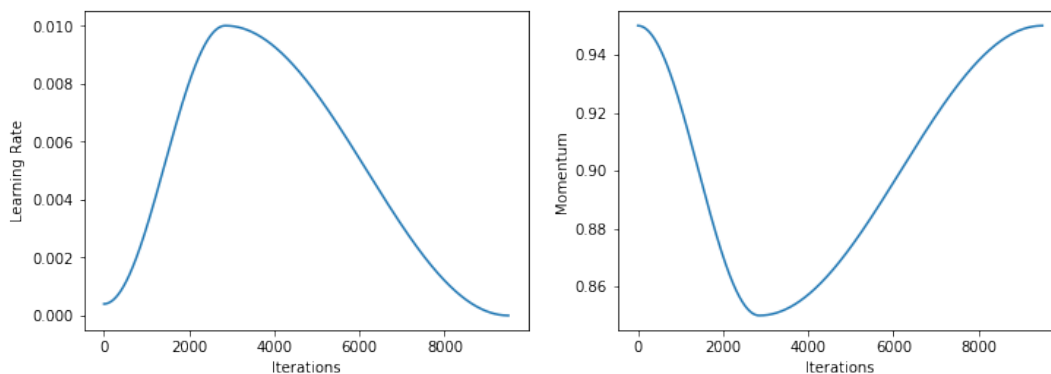
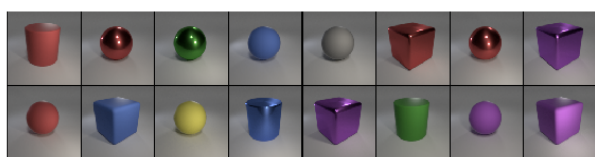
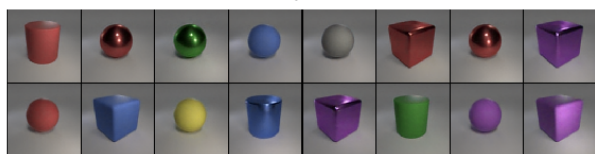


Figure 3: One-Cycle-Learning. Automated increasing and decreasing learning rates and corresponding momentum adaptation.



(a) Originals

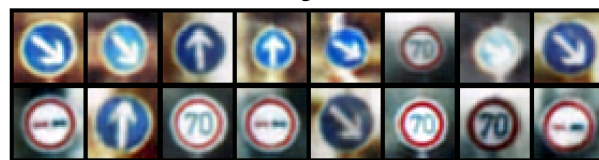


(b) Autoencoder reconstructions

Figure 4: A subset of images from the Non-Redundant-Objects data set.



(a) Originals



(b) Autoencoder reconstructions

Figure 6: A subset of images from the histogram equalized GTSRB data set.

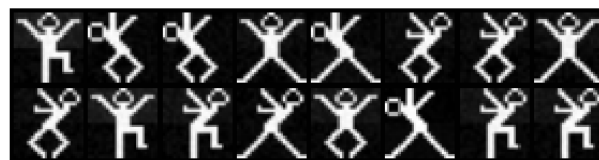


(a) Originals

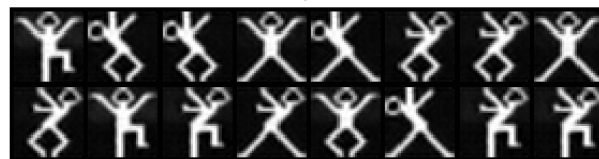


(b) Autoencoder reconstructions

Figure 5: A subset of images from the Concatenated-MNIST data set.



(a) Originals



(b) Autoencoder reconstructions

Figure 7: A subset of images from the stickfigures data set.

References

- Houben, S.; Stallkamp, J.; Salmen, J.; Schlipsing, M.; and Igel, C. 2013. Detection of traffic signs in real-world images: The german traffic sign detection benchmark. In *The 2013 international joint conference on neural networks IJCNN*, 1–8. IEEE.
- Howard, J., et al. 2018. fastai. <https://github.com/fastai/fastai>.
- Ioffe, S., and Szegedy, C. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32Nd International Conference on International Conference on Machine Learning - Volume 37*, ICML'15, 448–456. JMLR.org.
- Johnson, J.; Hariharan, B.; van der Maaten, L.; Fei-Fei, L.; Lawrence Zitnick, C.; and Girshick, R. 2017. Clevr: A diagnostic dataset for compositional language and elementary visual reasoning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR, 2017*, 2901–2910.
- Kingma, D. P., and Ba, J. 2014. Adam: A method for stochastic optimization. *CoRR* abs/1412.6980.
- LeCun, Y.; Bottou, L.; Bengio, Y.; Haffner, P.; et al. 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86(11):2278–2324.
- Nair, V., and Hinton, G. E. 2010. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ICML'10, 807–814. USA: Omnipress.
- Paszke, A.; Gross, S.; Chintala, S.; Chanan, G.; Yang, E.; DeVito, Z.; Lin, Z.; Desmaison, A.; Antiga, L.; and Lerer, A. 2017. Automatic differentiation in pytorch. In *NIPS-W*.
- Smith, L. N., and Topin, N. 2017. Super-convergence: Very fast training of residual networks using large learning rates. *ArXiv* abs/1708.07120.
- Smith, L. N. 2015. Cyclical learning rates for training neural networks. *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)* 464–472.
- Smith, L. N. 2018. A disciplined approach to neural network hyper-parameters: Part 1 - learning rate, batch size, momentum, and weight decay. *ArXiv* abs/1803.09820.
- Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; and Salakhutdinov, R. 2014. Dropout: A simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* 15(1):1929–1958.
- Ye, W.; Maurus, S.; Hubig, N.; and Plant, C. 2016. Generalized independent subspace clustering. In *IEEE 16th International Conference on Data Mining, ICDM 2016, December 12-15, 2016, Barcelona, Spain*, 569–578.